FPGA Based System Design

Title: I2C(INTER-INTEGRATED CIRCUIT) SLAVE

B.Tech Semester IV

Electronics and Communication Engineering Department

Submitted by

Mitanshu Dhameliya (23BEC042)

School of Engineering

Institute of Technology

Nirma University

Ahmedabad 382481

March 2025

Abstract

The Inter-Integrated Circuit (I2C) protocol is a widely used communication standard that facilitates data exchange between multiple integrated circuits (ICs) over a two-wire interface. the Inter-Integrated Circuit (I2C) protocol stands out for its efficiency in between microcontrollers and peripheral devices. By giving unique addresses, I2C slaves can engage in both transmit and receive operations. An I2C slave device is characterized by its unique address, which allows the master to identify and communicate with it specifically. The protocol supports multiple slave devices on the same bus, enabling efficient multidevice communication. Key features of I2C slave devices include data acknowledgment, clock stretching, and the ability to handle both read and write operations. Furthermore, it discusses the challenges and opportunities in implementing I2C slave functionality, particularly in scenarios requiring real-time operating systems or hardware based solutions

State of the art technology available

1. Recent Developments in I2C Technology

- **High-Speed Mode:-** Recent advancements have introduced high-speed modes (up to 3.4 Mbps) to enhance data transfer rates, catering to applications requiring faster communication.
- **Extended Addressing:-** Newer versions of the I2C protocol support extended addressing, allowing for more than 128 devices on the same bus, which is beneficial for large-scale applications.

2. Integration with FPGA and Microcontrollers

- The implementation of I2C controllers on FPGAs has gained traction due to their flexibility and reconfigurability. This allows for custom designs tailored to specific application needs.
- Microcontrollers with built-in I2C interfaces have become prevalent, simplifying the design process and reducing the need for external components.

3. I2C in IoT Applications

- The rise of the Internet of Things (IoT) has led to increased use of I2C for connecting sensors and actuators in smart devices. Its low pin count and simplicity make it ideal for resource-constrained environments.
- Research has focused on optimizing I2C communication for low-power applications, ensuring efficient energy consumption in battery-operated devices.

4. Comparative Studies with Other Protocols

- Comparative analyses between I2C and other communication protocols like SPI (Serial Peripheral Interface) and UART (Universal Asynchronous Receiver-Transmitter) highlight I2C's advantages in terms of wiring simplicity and device addressing, while also discussing its limitations in speed and distance.

Limitations or drawbacks of currently available technology

1. Speed Limitations

- **Data Transfer Rate:-** The standard I2C protocol operates at speeds of up to 400 Kbps in Fast Mode and 1 Mbps in Fast Mode Plus. While high-speed mode allows for up to 3.4 Mbps, this is still significantly slower compared to other protocols like SPI, which can achieve much higher data rates.
- **Latency:-** The inherent latency in the acknowledgment process can affect the overall speed of communication, especially in systems with multiple devices.

2. Distance Constraints

- **Short Communication Range:-** I2C is designed for short-distance communication, typically within a single board or between closely located devices. The maximum bus length is limited to about 1 meter, which can be a drawback in larger systems.
- **Signal Integrity:-** As the distance increases, the risk of signal degradation and noise interference also increases, potentially leading to communication errors.

3. Limited Number of Devices

- **Addressing Limitations:-** The standard I2C protocol supports a maximum of 128 devices on the bus due to its 7-bit addressing scheme. Although extended addressing (10-bit) is available, it is not widely adopted, limiting scalability in larger systems.

4. Power Consumption

- **Pull-Up Resistors:-** The requirement for pull-up resistors on the SDA and SCL lines can lead to increased power consumption, particularly in larger networks with many devices.

Proposed solution/methodology :-

- **Extended Addressing Support:-** Promote the use of 10-bit addressing to increase the number of devices that can be connected to the I2C bus, allowing for scalability in larger systems.

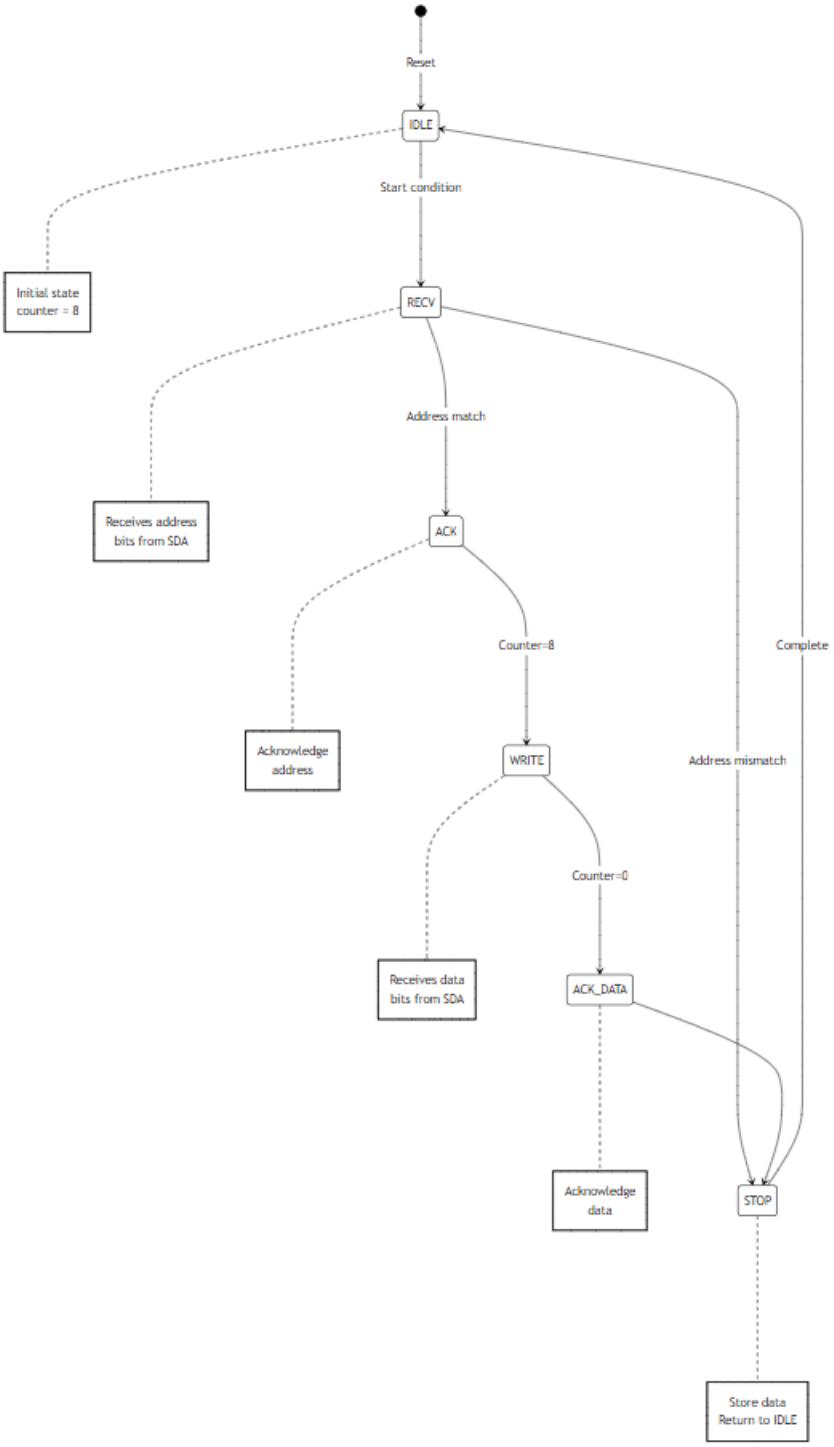
- **Adoption of High-Speed I2C:-** Utilize the high-speed mode of I2C (up to 3.4 Mbps) for applications requiring faster data transfer rates. This can be achieved by ensuring that all devices support high-speed operation.

- **Support for Complex Data Types:-** Develop extensions to the I2C protocol that allow for the transmission of complex data types, such as structures or arrays, to accommodate a wider range of applications.

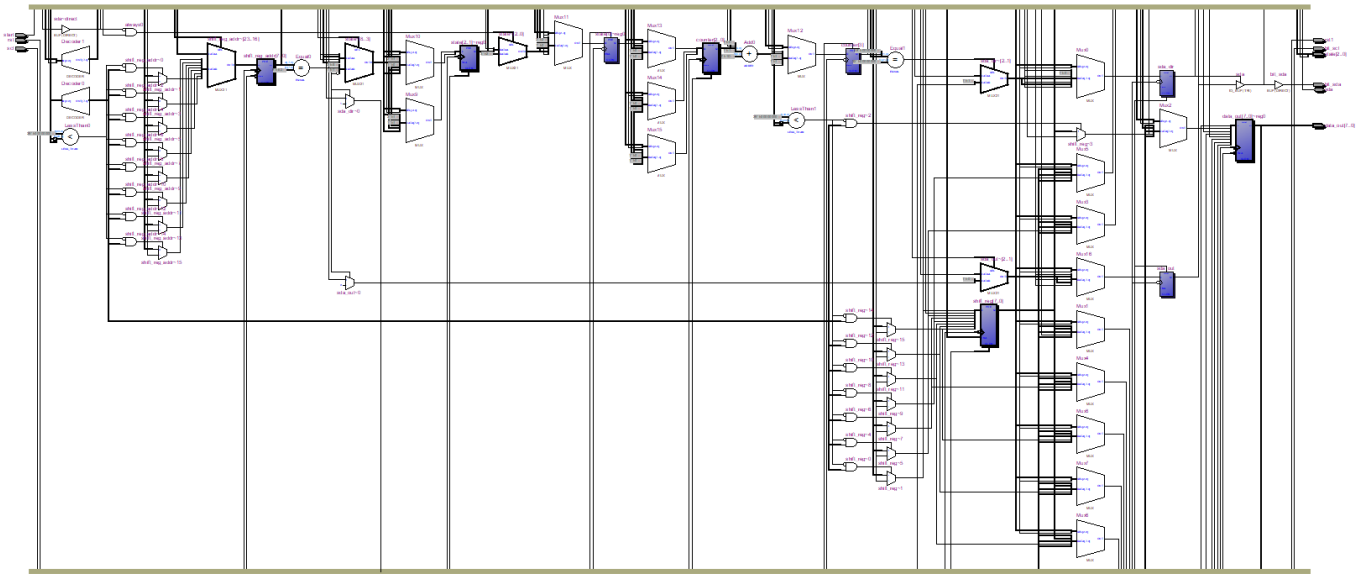
- **Advanced Simulation Tools:-** Utilize advanced simulation tools to model and test I2C implementations under various conditions, allowing for the identification and resolution of potential issues before deployment.

- **Real-Time Monitoring Solutions:-** Implement real-time monitoring solutions to track the performance of I2C communication in live systems, enabling proactive management and troubleshooting.

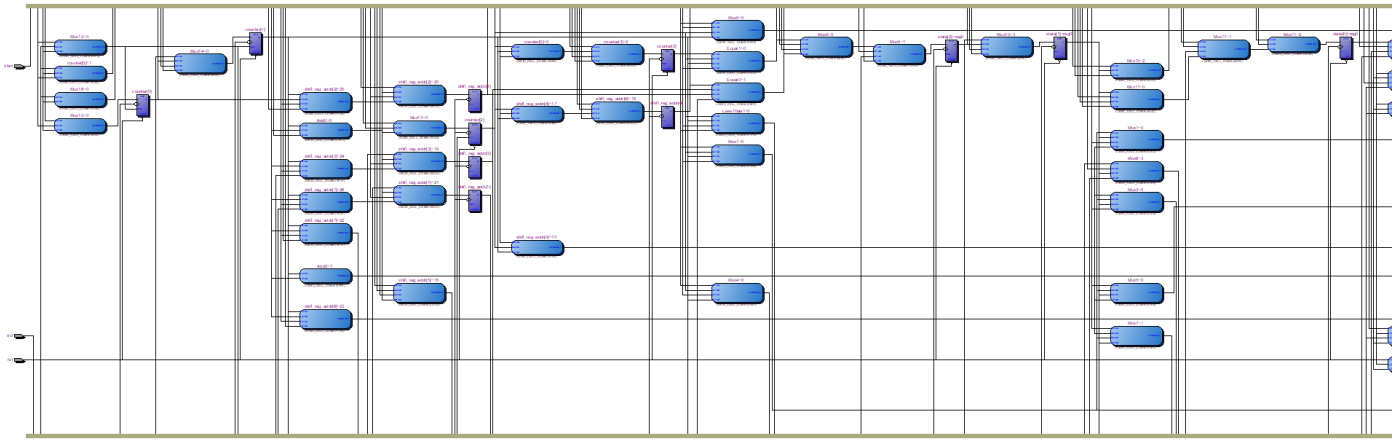
Flow chart

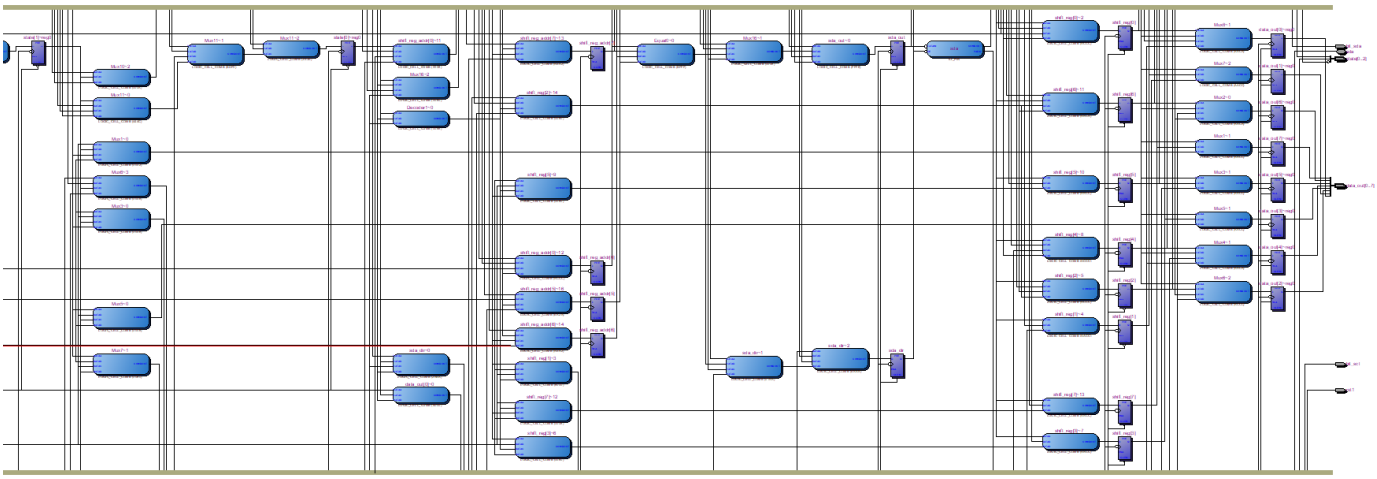

Reset
IDLE
Start condition
Initial state
counter = 8
RECV
Address match
Receives address
bits from SDA
ACK
Counter=8
Complete
Acknowledge
address
WRITE
Address mismatch
Counter=0
Receives data
bits from SDA
ACK_DATA
Acknowledge
data
STOP
Store data
Return to IDLE

RTL view

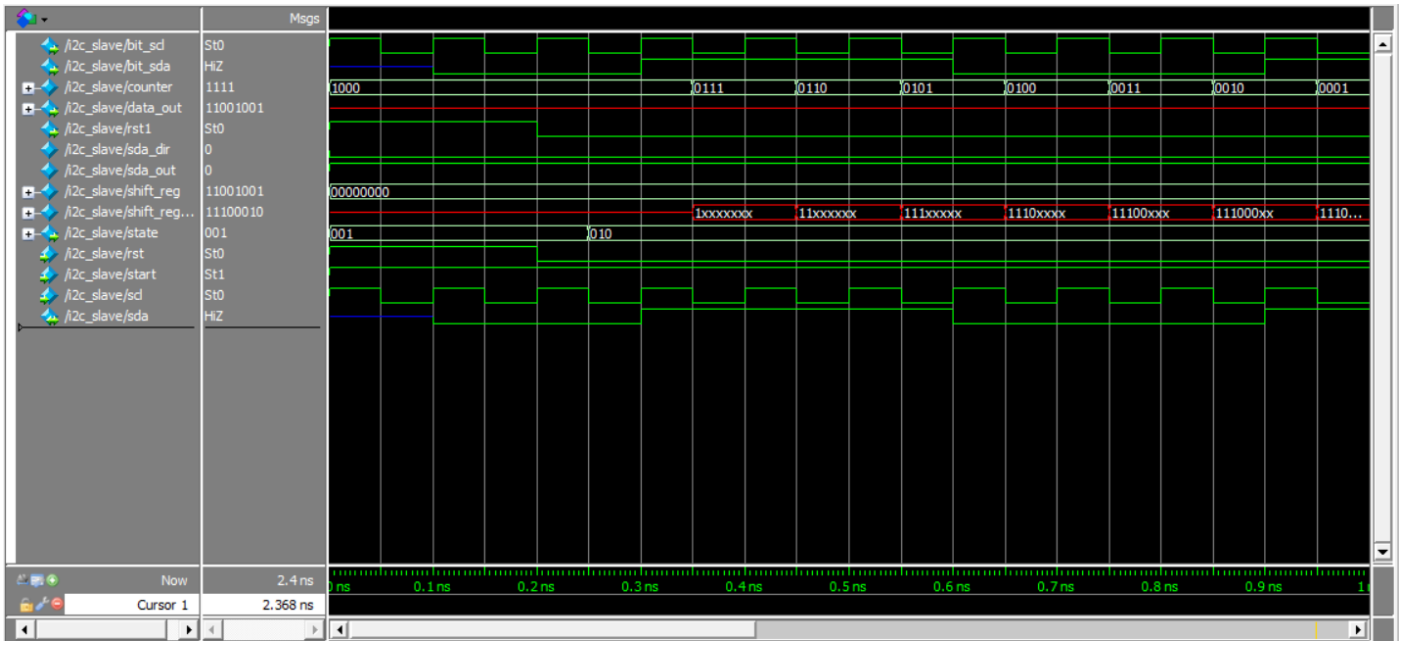


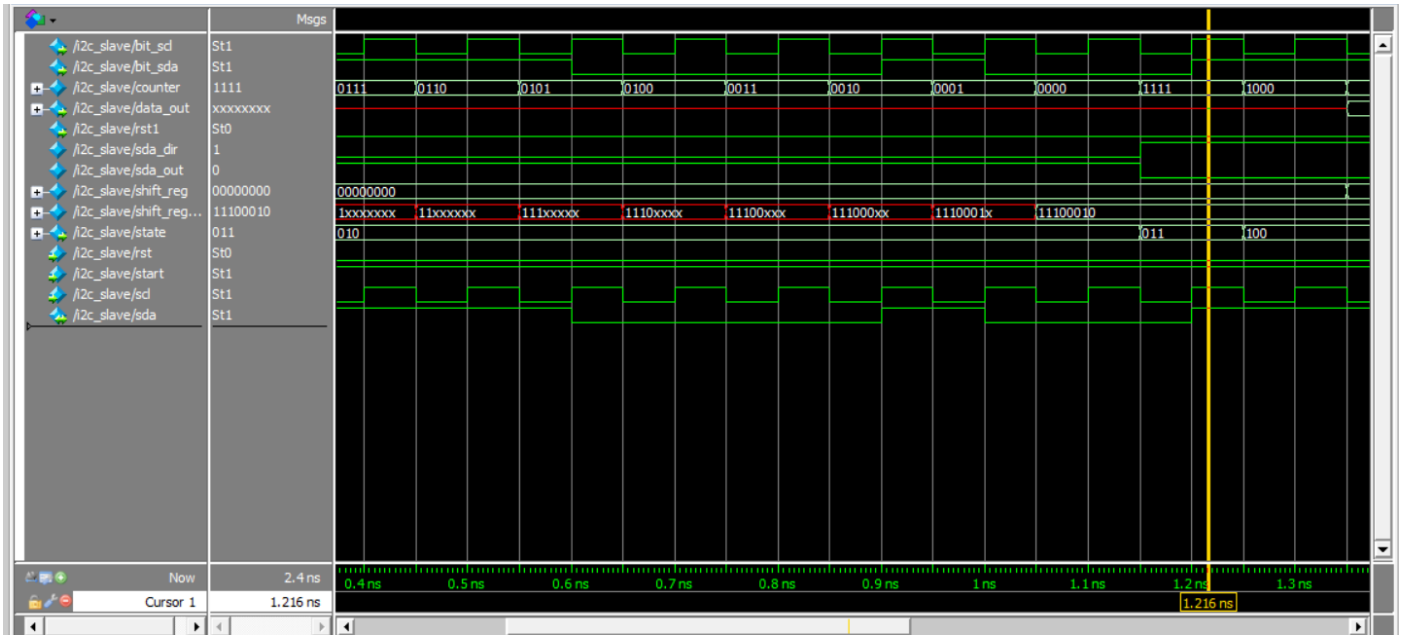
TTL view

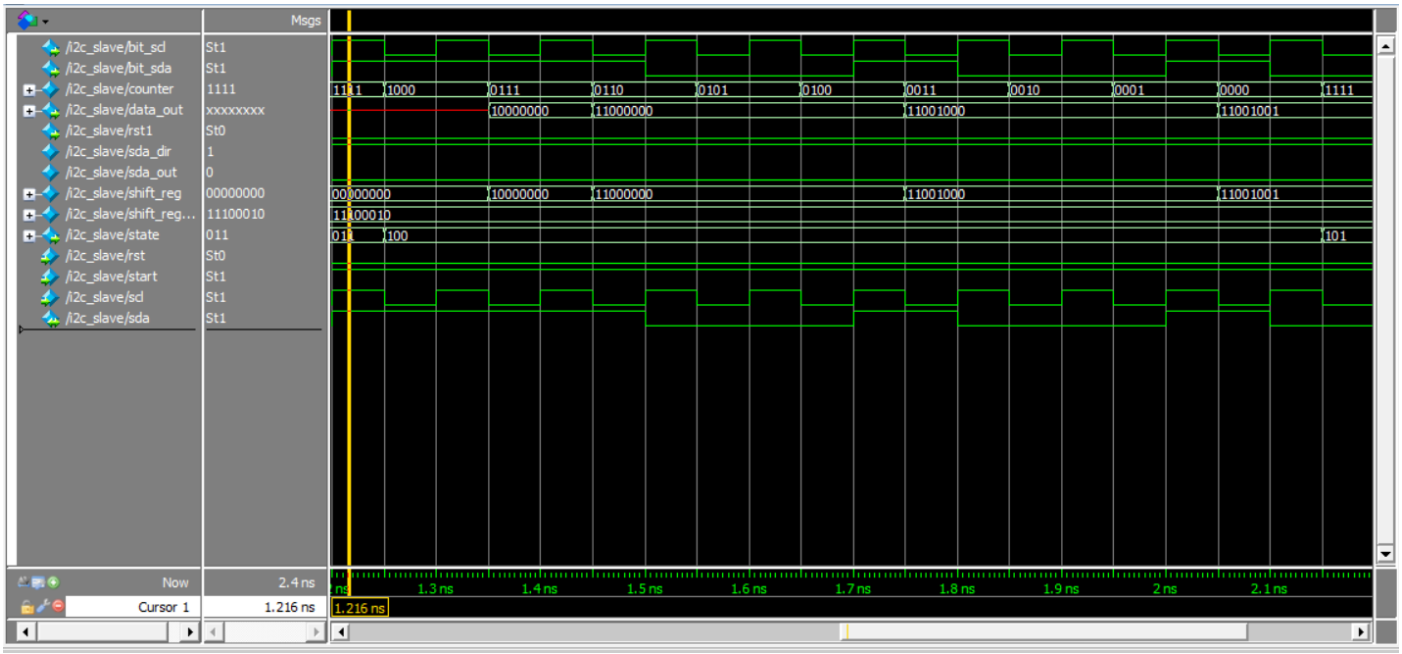


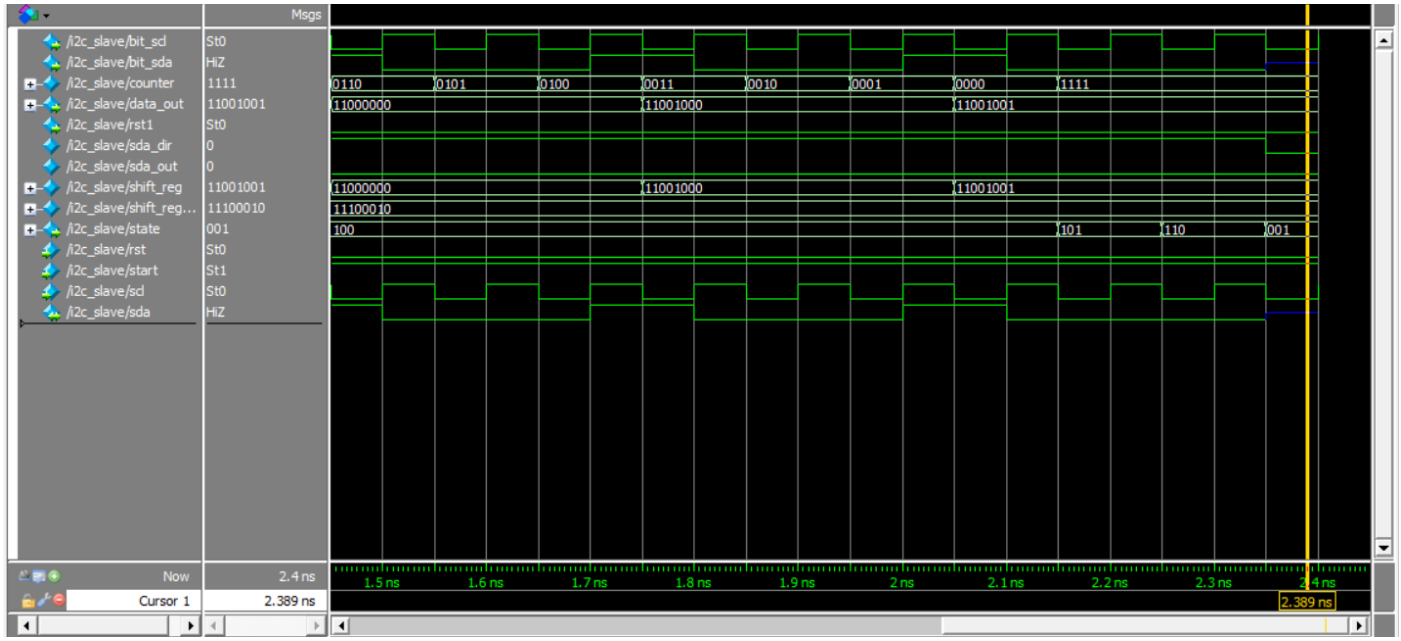


Simulation Waveforms




Msgs
/i2c_slave/bit_scl
/i2c_slave/bit_sda
/i2c_slave/counter
/i2c_slave/data_out
/i2c_slave/rst1
/i2c_slave/sda_dir
/i2c_slave/sda_out
/i2c_slave/shift_reg
/i2c_slave/shift_reg...
/i2c_slave/state
/i2c_slave/rst
/i2c_slave/start
/i2c_slave/scl
/i2c_slave/sda
St1
St1
1111
xxxxxxxx
St0
1
0
00000000
11100010
011
St0
St1
St1
St1
0111 0110 0101 0100 0011 0010 0001 0000 1111 1000
00000000
1xxxxxxx 11xxxxxx 111xxxxx 1110xxxx 11100xxx 111000xx 1110001x 11100010
010 011 100
Now
2.4 ns
Cursor 1
1.216 ns
0.4 ns 0.5 ns 0.6 ns 0.7 ns 0.8 ns 0.9 ns 1 ns 1.1 ns 1.2 ns 1.3 ns
1.216 ns


Msgs
/i2c_slave/bit_scl
/i2c_slave/bit_sda
/i2c_slave/counter
/i2c_slave/data_out
/i2c_slave/rst1
/i2c_slave/sda_dir
/i2c_slave/sda_out
/i2c_slave/shift_reg
/i2c_slave/shift_reg...
/i2c_slave/state
/i2c_slave/rst
/i2c_slave/start
/i2c_slave/scl
/i2c_slave/sda
St1
St1
1111
xxxxxxxx
St0
1
0
00000000
11100010
011
St0
St1
St1
St1
1111 1000 0111 0110 0101 0100 0011 0010 0001 0000 1111
10000000 11000000 11001000 11001001
00000000 10000000 11000000 11001000 11001001
11100010
011 100 101
Now
2.4 ns
Cursor 1
1.216 ns
1.3 ns 1.4 ns 1.5 ns 1.6 ns 1.7 ns 1.8 ns 1.9 ns 2 ns 2.1 ns
1.216 ns


Msgs
/i2c_slave/bit_scl
/i2c_slave/bit_sda
/i2c_slave/counter
/i2c_slave/data_out
/i2c_slave/rst1
/i2c_slave/sda_dir
/i2c_slave/sda_out
/i2c_slave/shift_reg
/i2c_slave/shift_reg...
/i2c_slave/state
/i2c_slave/rst
/i2c_slave/start
/i2c_slave/scl
/i2c_slave/sda
St0
HiZ
1111
11001001
St0
0
0
11001001
11100010
001
St0
St1
St0
HiZ
0110 0101 0100 0011 0010 0001 0000 1111
11000000 11001000 11001001
11000000 11001000 11001001
11100010
100 101 110 001
Now
2.4 ns
Cursor 1
2.389 ns
1.5 ns 1.6 ns 1.7 ns 1.8 ns 1.9 ns 2 ns 2.1 ns 2.2 ns 2.3 ns 2.4 ns
2.389 ns

Conclusion

This Verilog code implements an I2C slave controller on an FPGA using a well-structured state machine, ensuring efficient serial communication. It effectively handles key I2C functions, including data transfer, slave address transmission, start and stop condition generation, and acknowledgment management. To maintain precise timing, the design incorporates a clock division mechanism that generates a suitable clock signal for I2C operations. The use of tri-state logic on the SDA line enables proper bidirectional communication, while state management ensures smooth transitions between different phases of the I2C transaction. Overall, this implementation provides a reliable foundation for integrating I2C communication into embedded systems, facilitating stable and efficient interaction with multiple devices.

References

1)https://github.com/jiacaiyuan/i2c_slave

2)https://stackoverflow.com/questions/53451470/i2c-slave-module-in-verilogdoes-not-acknowledge

Appendix

```
module i2c_slave (
    input wire scl,
    input wire rst,
    inout wire sda,
    output reg [7:0] data_out,
   output reg [2:0] state,
   output  bit_scl,
 output  bit_sda,
 output rst1,
 input start
);

    //reg [2:0] state;
    reg [2:0] bit_count;
    reg [3:0] counter;
    reg sda_dir;
    reg sda_out;
    reg [7:0] shift_reg;
    reg [7:0] shift_reg_addr;
    parameter address = 8'b11100010;


    assign sda = sda_dir ? sda_out : 1'bz;

    localparam IDLE  = 3'b001,
               RECV  = 3'b010,
               ACK   = 3'b011,
               WRITE = 3'b100,
               ACK_DATA = 3'b101,
               STOP  = 3'b110;

    always @(negedge scl or posedge rst) begin
        if (rst)
        begin
            state <= IDLE;
            bit_count <= 0;
            sda_dir <= 0;
            shift_reg <= 0;
            sda_out <= 1;
            counter <= 8;
        end
        else
```

```
begin
   case (state)
      // Wait for START condition
      IDLE:
      begin
         if (sda==0 && start==1)
           begin
                counter <= 8;
                state <= RECV;  // Start receiving address/data
           end
      end

      // Receiving address or data
      RECV:
      begin
        shift_reg_addr[counter-1] = sda;
        counter <= counter -1;
        if(counter == 0)
              begin
                    if(address==shift_reg_addr)
                    begin
                          sda_dir <= 1;
                          sda_out <= 0;
                          state <= ACK;
                    end
                    else
                    state <= STOP;
             end

      end

      // Acknowledge received address
      ACK:
      begin
         counter <=  8;
         state <= WRITE; // Proceed to WRITE state
      end

      // Writing (Receiving data from master)
```

```
            WRITE:
            begin
                shift_reg[counter-1] = sda;
                counter <= counter -1;
                if(counter == 0)
                        begin
                        sda_dir <= 1;
                        sda_out <= 0;
                        state <= ACK_DATA;
                        end
            end

            // Acknowledge received data
            ACK_DATA:
            begin
               state <= STOP;
            end

            // Stop Condition
            STOP:
            begin
               sda_dir <= 0;
               data_out <= shift_reg; // Store received data
               state <= IDLE;
            end
          endcase
       end
   end


assign bit_sda = sda;
assign bit_scl = scl;
assign rst1 = rst;

endmodule
```